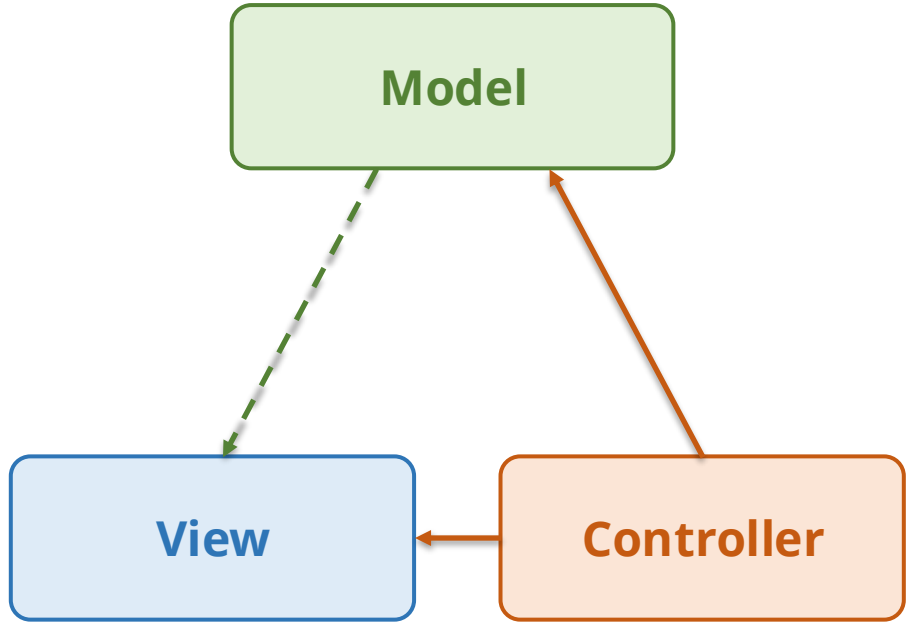
PUC Minas

Padrão MVC em Aplicações Web

Prof. Rommel Vieira Carneiro

Tópicos

- **Introdução e contextualização**
  - O que é o MVC e breve histórico
  - O problema do código acoplado e a solução
  - Vantagens e cuidados
- **Anatomia das três camadas**
  - Model, View e Controller no ecossistema Web
- **O ciclo de vida da requisição Web no MVC**
  - Roteamento, controller, model, view e resposta HTTP
- **Implementação prática com Express e EJS**
  - Ambiente, estrutura de pastas e CRUD de Produtos
- **MVC no contexto moderno**
  - SSR vs. SPA, MVC em APIs e no front-end, MVP e MVVM
  - Fat Model, Skinny Controller e camada de serviço



Bibliografia da Unidade

GAMMA, E. et al. Padrões de projeto: soluções reutilizáveis de software orientado a objetos. Bookman, 2000.
Bibliografia básica – o capítulo 1 descreve o MVC como uma combinação de padrões (Observer, Composite, Strategy).

FOWLER, M. Patterns of Enterprise Application Architecture. Addison-Wesley, 2002.
Referência complementar – padrões Model View Controller, Page/Front Controller, Template View, Domain Model e Service Layer.

REENSKAUG, T. The original MVC reports. Xerox PARC, 1978-79.
Documentos históricos do criador do padrão, disponíveis em: folk.universitetetioslo.no/trygver

Express / EJS Documentação oficial: expressjs.com e ejs.co
Referência técnica para os exemplos práticos desta unidade.

PUC Minas

Padrão MVC

Introdução e Contextualização

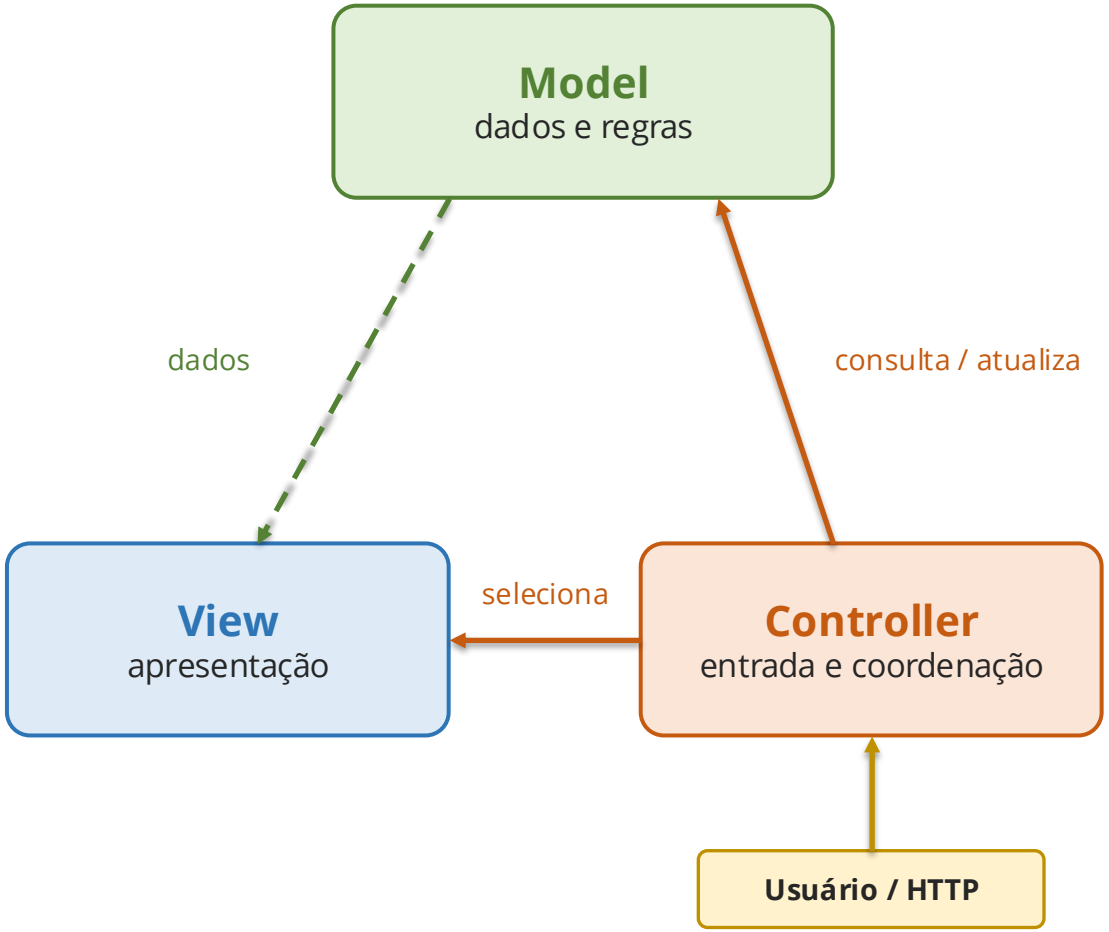
Prof. Rommel Vieira Carneiro

O que é o MVC

O **Model-View-Controller (MVC)** é um **padrão arquitetural** que organiza uma aplicação interativa em três papéis com responsabilidades distintas:

- **Model** – dados, estado e regras de negócio
- **View** – apresentação das informações ao usuário
- **Controller** – trata as entradas do usuário e coordena Model e View

O objetivo central é a **separação de responsabilidades**: cada parte muda por um motivo diferente e pode evoluir de forma independente.



Padrão de projeto ou padrão arquitetural?

	Padrão de projeto (design pattern)	Padrão arquitetural
Escopo	Classes e objetos que colaboram	Organização da aplicação como um todo
Granularidade	Fina – resolve um problema local de design	Grossa – define camadas, papéis e fluxos
Exemplos	Observer, Strategy, Composite, Factory	MVC, Camadas, Microsserviços, Event-Driven
Referência	Gamma et al. (GoF, 1994)	Buschmann et al. (POSA, 1996); Fowler (2002)

O MVC é uma combinação de padrões de projeto

No livro do GoF, o MVC aparece como exemplo motivador: a View observa o Model (**Observer**), Views podem ser compostas de outras Views (**Composite**) e a View delega o tratamento de entradas a um Controller intercambiável (**Strategy**).

Fonte: GAMMA, E. et al. Padrões de projeto. Bookman, 2000 – Cap. 1.2 "Padrões de projeto no MVC de Smalltalk".

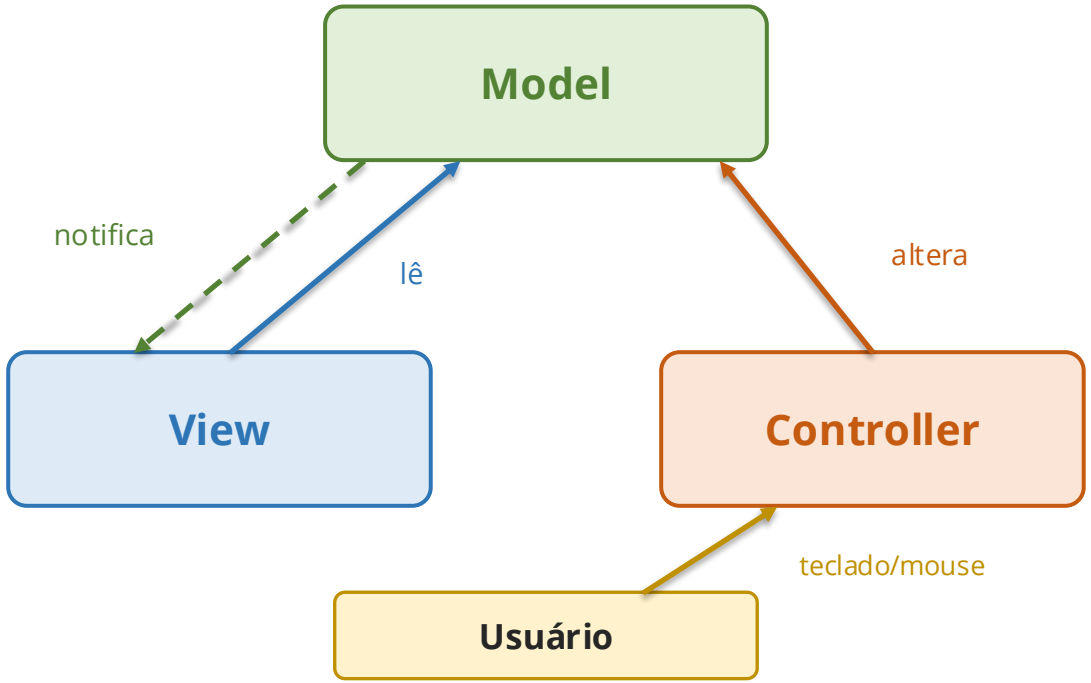
Histórico do MVC

1979	Trygve Reenskaug cria o MVC no Xerox PARC para o Smalltalk-79
1988	Krasner & Pope publicam o “cookbook” do MVC no Smalltalk-80
1994	GoF descreve o MVC como combinação de padrões de projeto
1996	Taligent (IBM) propõe o MVP – Model-View-Presenter
1999	JSP Model 2 leva o MVC para a Web (Servlet + JSP)
2004	Ruby on Rails e Spring MVC popularizam o MVC no servidor
2005	Microsoft apresenta o MVVM – Model-View-ViewModel (WPF)
2010	Backbone.js e AngularJS levam o padrão ao navegador
2013	React: foco na View e em componentes; MVC vira API no back-end

Fonte: REENSKAUG, T. The original MVC reports, 1979; KRASNER, G.; POPE, S. JOOP, 1988; POTEL, M. MVP, Taligent, 1996; GOSSMAN, J. Introduction to MVVM, 2005.

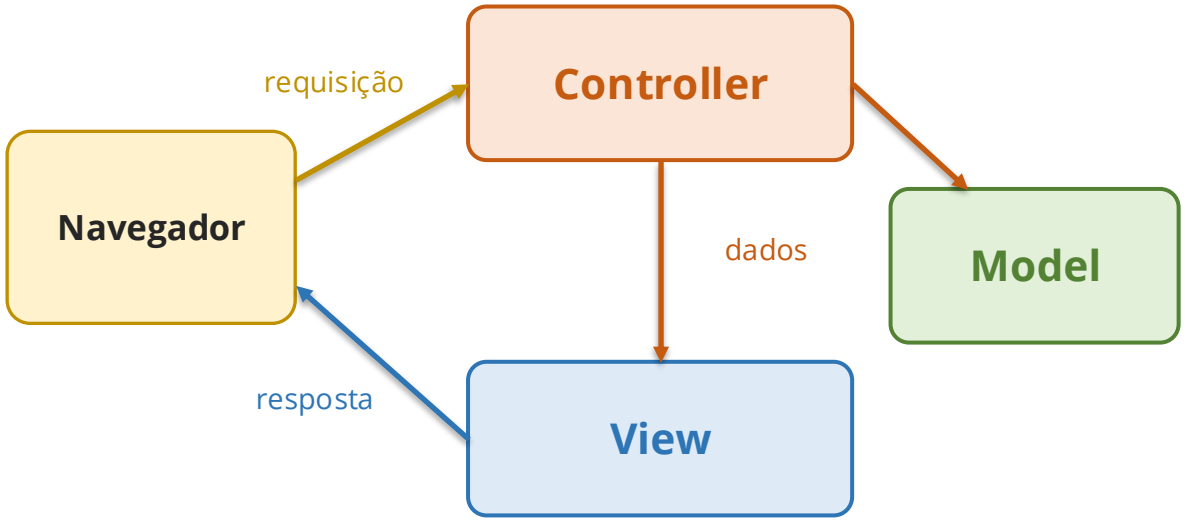
MVC clássico vs. MVC na Web

MVC clássico (aplicações desktop)



O Model **notifica** as Views (Observer) sempre que muda – todos convivem na mesma memória.

MVC na Web (Model 2)



O HTTP é **sem estado**: não há notificação do Model para a View. A cada requisição o Controller consulta o Model, entrega os dados à View e a resposta é enviada ao navegador.

O Problema: código acoplado (“espaguete”)

app.js

```
app.get('/produtos', (req, res) => {
  const produtos = JSON.parse(fs.readFileSync('produtos.json'));
  let html = '<html><body><h1>Produtos</h1><ul>';
  for (const p of produtos) {
    const preco = p.estoque > 10 ? p.preco * 0.9 : p.preco;
    html += `<li>${p.nome} - R$ ${preco.toFixed(2)}</li>`;
  }
  html += '</ul></body></html>';
  res.send(html);
});
```

1. **Acesso a dados**
   leitura do arquivo dentro do tratamento da rota
2. **Regra de negócio**
   desconto calculado no meio da montagem do HTML
3. **Apresentação**
   HTML construído por concatenação de strings
4. **Segurança**
   dados inseridos sem escape → risco de XSS

Tudo funciona... até a primeira mudança. Um único trecho mistura **quatro motivos diferentes para mudar** – e qualquer alteração em um deles arrisca quebrar os outros.

Consequências do acoplamento

Manutenção cara

Trocar o layout exige mexer em código que também contém regras de negócio.

Duplicação

A mesma regra (ex.: desconto) é copiada na página HTML, na API e no relatório.

Testes difíceis

Não é possível testar a regra de desconto sem subir o servidor e fazer uma requisição.

Conflitos na equipe

Designer e desenvolvedor alteram o mesmo arquivo → conflitos constantes no Git.

A Solução: separar responsabilidades

Model – regra de negócio

```
class Produto {
  get disponivel() {
    return this.estoque > 0;
  }
}
```

Controller – coordenação

```
function listar(req, res) {
  const produtos = repo.listar();
  res.render('produtos/index',
             { produtos });
}
```

View – apresentação

```
<% produtos.forEach(p => { %>
  <li><%= p.nome %></li>
<% }) %>
```

Cada parte tem **um único motivo para mudar**:

- a regra “produto disponível” vive no Model e é reaproveitada por qualquer View ou API;
- a View só exibe – o EJS escapa automaticamente os dados com `<%= %>`;
- o Controller apenas liga a requisição ao Model e escolhe a View.

Vantagens do MVC

Separação de conceitos

Cada camada trata de um único assunto (Separation of Concerns).

Manutenção facilitada

Mudanças ficam localizadas: layout na View, regra no Model.

Testabilidade

Regras do Model são testadas isoladamente, sem HTTP nem HTML.

Desenvolvimento paralelo

Front-end, back-end e dados evoluem em frentes simultâneas.

Reuso

O mesmo Model atende várias Views: página HTML, API JSON, relatório.

Organização padronizada

Convenção conhecida por equipes e frameworks de mercado.

Cuidados: o MVC não é bala de prata

- **Mais arquivos e indireção** – para uma página simples, a separação pode parecer excessiva.
- **Controllers inchados** – sem disciplina, a regra de negócio migra para o Controller e o problema do "espaguete" volta.
- **Fronteiras mal compreendidas** – "o que é Model?" gera discussões; o padrão não define como organizar regras complexas.
- **Não resolve tudo** – regras que envolvem vários objetos pedem camadas adicionais (serviços, repositórios).



Regra prática

Use o MVC como ponto de partida e evolua a arquitetura conforme a complexidade do domínio.

Veremos isso em "Fat Model, Skinny Controller" e na camada de serviço.

PUC Minas

Padrão MVC

Anatomia das Três Camadas

Prof. Rommel Vieira Carneiro

Visão geral das camadas

Model

Responsável por

- Representa dados e estado
- Aplica regras de negócio
- Valida informações
- Acessa a persistência (via repositório)

Não deve

- Conhecer HTTP (req/res)
- Gerar HTML ou JSON

View

Responsável por

- Exibe dados ao usuário
- Formata valores (moeda, data)
- Estrutura HTML/CSS
- Laços e condicionais de exibição

Não deve

- Calcular regras de negócio
- Consultar banco de dados

Controller

Responsável por

- Recebe a requisição HTTP
- Extrai parâmetros e formulários
- Aciona o Model
- Escolhe View, status ou redirect

Não deve

- Conter regras de negócio
- Montar HTML manualmente

Model (Modelo)



O Model representa o **domínio** da aplicação: os conceitos do negócio, seus dados e as regras que os governam.

- **Dados e estado** – atributos do produto (nome, preço, estoque)
- **Regras de negócio** – "desconto máximo de 50%", "produto sem estoque não pode ser vendido"
- **Validações** – garantir que um produto nunca exista em estado inválido
- **Independente da interface** – não conhece `req`, `res`, HTML ou JSON

models/produto.js (trecho)

```
export class Produto {
  constructor({ id, nome, descricao, preco, estoque, categoria }) {
    this.id = id;
    this.nome = nome?.trim();
    this.descricao = descricao?.trim() || null;
    this.preco = numeroOuNulo(preco);
    this.estoque = numeroOuNulo(estoque);
    this.categoria = categoria || 'Geral';
  }

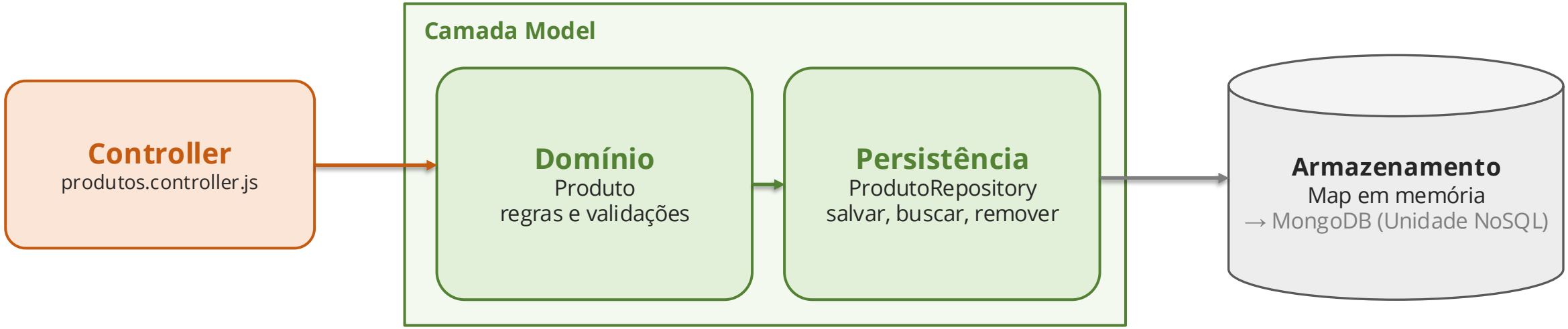
  // errosPorCampo() e validar(): regras de validação

  get disponivel() {
    return this.estoque > 0;
  }

  precoComDesconto(percentual) {
    if (percentual < 0 || percentual > 50)
      throw new Error('Desconto deve estar entre 0% e 50%');
    return Number((this.preco * (1 - percentual / 100)).toFixed(2));
  }
}
```

Model – Domínio vs. Persistência

Um erro comum é tratar “Model” como sinônimo de “tabela do banco”. O Model é o **domínio**; a **persistência** é apenas um dos seus detalhes de implementação.



- **Domínio**: o que é um produto válido e o que pode acontecer com ele.
- **Persistência**: onde e como os produtos são guardados. Trocar o armazenamento não deve alterar as regras.

Model – Repositório (persistência)

models/produto.repository.js

```
// Persistência dos produtos (em memória, por enquanto)
import { Produto } from './produto.js';
import { CategoriaRepository } from './categoria.repository.js';

const produtos = new Map();
let proximoId = 1;

const porNome = (a, b) => a.nome.localeCompare(b.nome, 'pt-BR');

export const ProdutoRepository = {
  listar: () => [...produtos.values()],
  buscarPorId: (id) => produtos.get(Number(id)) ?? null,
  salvar(produto) {
    produto.id ??= proximoId++;
    CategoriaRepository.garantir(produto.categoria);
    produtos.set(produto.id, produto);
    return produto;
  },
  remover: (id) => produtos.delete(Number(id)),
  // ... pesquisar(): filtros e paginação (usado pela API)
};
```

O **repositório** isola o acesso aos dados em uma interface simples: `listar`, `buscarPorId`, `salvar` e `remover`.

Nesta unidade os dados ficam em memória (`Map`) e são perdidos ao reiniciar o servidor.

Na unidade de NoSQL, a implementação passa a usar MongoDB e Mongoose – Controller e regras de negócio **não mudam**.

View (Visão)



A View é a **interface do usuário**: transforma os dados recebidos em uma representação – HTML e CSS para o navegador ou JSON para outras aplicações.

Na View pode	Na View não pode
• Laços para listar itens • Condicionais de exibição (mostrar/ocultar) • Formatação de moeda, datas e textos • Inclusão de trechos reutilizáveis (partials)	• Calcular preços, descontos ou impostos • Validar regras de negócio • Consultar o banco ou chamar o repositório • Ler `req` ou decidir redirecionamentos

Quanto mais "burra" a View, mais fácil trocá-la – por outro layout, por um app mobile ou por uma API.

View – Template engine EJS

Um **template engine** combina um modelo de página (HTML) com dados para gerar a resposta. O **EJS** (Embedded JavaScript) usa JavaScript puro dentro de marcadores especiais:

Marcador	Função	Exemplo
`<% %>`	Executa código sem gerar saída (laços, condicionais)	`<% if (erros.length) { %>`
`<%= %>`	Imprime o valor **escapando HTML** (seguro contra XSS)	`<%= produto.nome %>`
`<%- %>`	Imprime **sem escape** – usar só com conteúdo confiável	`<%- include('partials/rodape') %>`
`<%# %>`	Comentário – não aparece na resposta	`<%# lista de produtos %>`

Atenção à segurança
Dados vindos do usuário devem ser exibidos sempre com `<%= %>`. Usar `<%- %>` com esses dados abre espaço para ataques de Cross-Site Scripting (XSS).

Fonte: EJS – Embedded JavaScript templates. https://ejs.co

View – Exemplo: listagem de produtos

views/produtos/index.ejs

```
<%- include('../partials/cabecalho') %>
<h1>Produtos</h1>
<a href="/produtos/novo">+ Novo produto</a>

<table>
  <tr><th>Nome</th><th>Categoria</th><th>Preço</th><th>Situação</th></tr>
  <% produtos.forEach(p => { %>
    <tr>
      <td><a href="/produtos/<%= p.id %>"><%= p.nome %></a></td>
      <td><%= p.categoria %></td>
      <td><%= p.preco.toLocaleString('pt-BR',
                { style: 'currency', currency: 'BRL' }) %></td>
      <td><%= p.disponivel ? 'Em estoque' : 'Esgotado' %></td>
    </tr>
  <% }) %>
</table>
<%- include('../partials/rodape') %>
```

- Os dados (`produtos`) chegam prontos do Controller.
- `forEach` apenas percorre a lista – lógica de **exibição**.
- `toLocaleString` formata a moeda – decisão de **apresentação**.
- `p.disponivel` é uma regra do **Model**: a View apenas pergunta, não calcula.

View – Exibir vs. decidir

✗ View com regra de negócio

```
<% const preco = p.estoque > 10
     ? p.preco * 0.9 : p.preco; %>
<% if (p.estoque > 0 && p.ativo) { %>
  <td>R$ <%= preco.toFixed(2) %></td>
  <td>Em estoque</td>
<% } %>
```

- Regra de desconto e de disponibilidade **duplicadas** em cada página que exibe produtos.
- A API JSON precisará repetir o mesmo cálculo.

✓ View apenas exibe

```
<td><%= p.preco.toLocaleString('pt-BR',
      { style: 'currency', currency: 'BRL' }) %></td>
<td><%= p.disponivel ? 'Em estoque'
                     : 'Esgotado' %></td>
```

- A regra fica no Model (`get disponivel()`) e vale para toda a aplicação.
- A View só decide **como mostrar**: texto, cor, formato.

View – Partials (componentes reutilizáveis)

views/partials/cabecalho.ejs

```ejs
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title><%= titulo %> | Loja DIAW</title>
  <link rel="stylesheet" href="/css/estilo.css">
</head>
<body>
  <header><a href="/produtos">Loja DIAW</a></header>
  <main>
```

views/partials/rodape.ejs

```ejs
  </main>
  <footer>DIAW – Exemplo MVC com Express e EJS</footer>
</body>
</html>
```

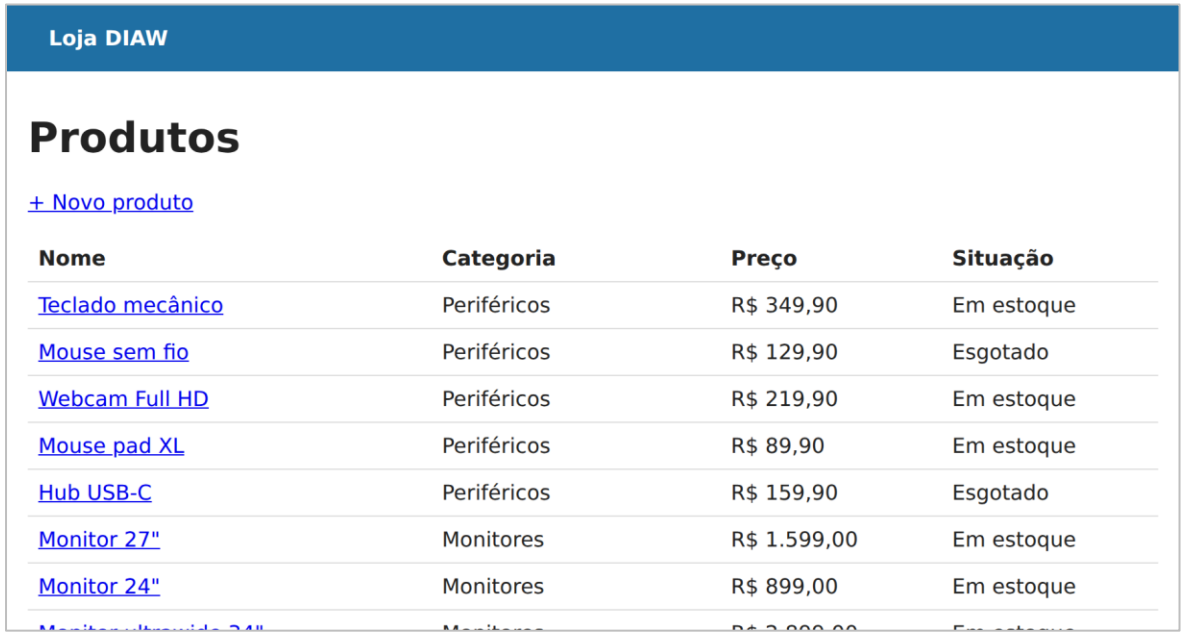
views/404.ejs

```ejs
<%- include('partials/cabecalho') %>
<h1><%= titulo %></h1>
<a href="/produtos">Voltar para a lista de produtos</a>
<%- include('partials/rodape') %>
```

- **Partials** são fragmentos de View incluídos em várias páginas com `include`.
- Evitam repetir cabeçalho, menu e rodapé.
- A variável `titulo`, enviada pelo Controller, é compartilhada com o partial.

A View não é só HTML

O **mesmo Model** pode ter **várias Views**. Abaixo, os mesmos produtos em duas representações diferentes:



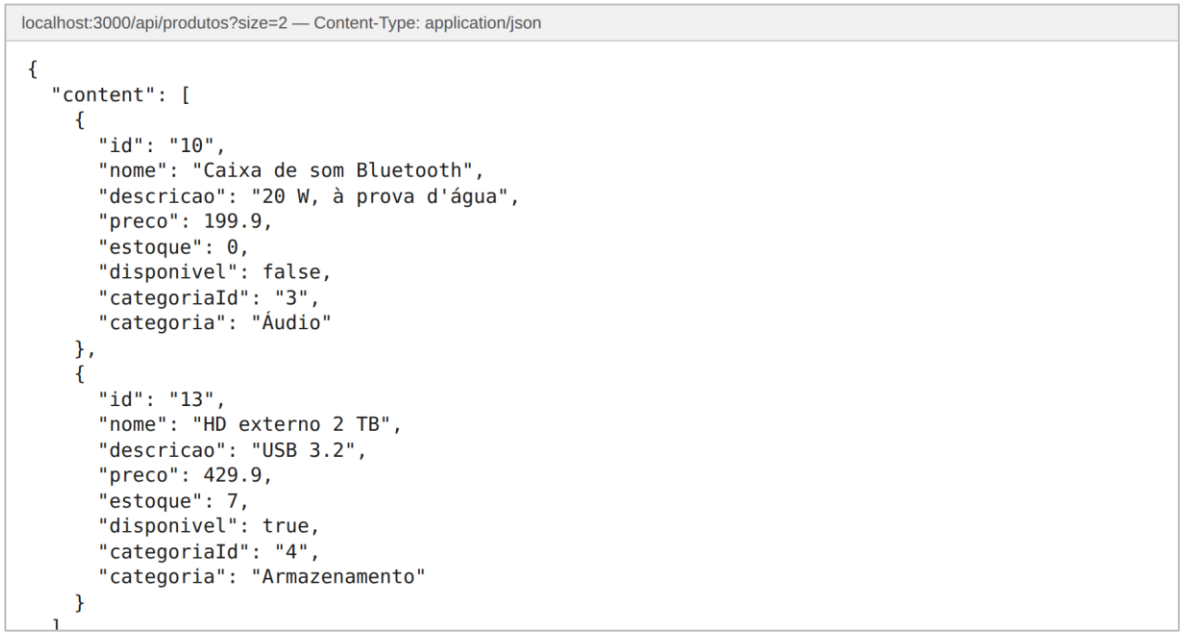
Loja DIAW

Produtos

+ Novo produto

Nome	Categoria	Preço	Situação
Teclado mecânico	Periféricos	R$ 349,90	Em estoque
Mouse sem fio	Periféricos	R$ 129,90	Esgotado
Webcam Full HD	Periféricos	R$ 219,90	Em estoque
Mouse pad XL	Periféricos	R$ 89,90	Em estoque
Hub USB-C	Periféricos	R$ 159,90	Esgotado
Monitor 27"	Monitores	R$ 1.599,00	Em estoque
Monitor 24"	Monitores	R$ 899,00	Em estoque

GET /produtos → HTML gerado pelo EJS



localhost:3000/api/produtos?size=2 — Content-Type: application/json


```
{
  "content": [
    {
      "id": "10",
      "nome": "Caixa de som Bluetooth",
      "descricao": "20 W, à prova d'água",
      "preco": 199.9,
      "estoque": 0,
      "disponivel": false,
      "categoriaId": "3",
      "categoria": "Áudio"
    },
    {
      "id": "13",
      "nome": "HD externo 2 TB",
      "descricao": "USB 3.2",
      "preco": 429.9,
      "estoque": 7,
      "disponivel": true,
      "categoriaId": "4",
      "categoria": "Armazenamento"
    }
  ]
```


GET /api/produtos?size=2 → JSON gerado por res.json()

Controller (Controlador)



O Controller é o **coordenador** da requisição: recebe a entrada HTTP, aciona o Model e decide qual resposta será enviada. Ele **orquestra**, mas não decide regras de negócio.

1. **Recebe**
   a requisição mapeada pelo roteador
2. **Extrai**
   parâmetros da URL e dados do formulário
3. **Aciona**
   o Model para consultar ou alterar dados
4. **Responde**
   com uma View, um status ou um redirecionamento

Analogia: o maestro
O maestro não toca os instrumentos (regras = Model) nem é a plateia que vê o espetáculo (View). Ele indica quem entra e quando.

Controller – Exemplo

controllers/produtos.controller.js

```
// Controller: recebe a requisição, aciona o Model e escolhe a View
import { Produto } from '../models/produto.js';
import { ProdutoRepository as repo } from '../models/produto.repository.js';

const naoEncontrado = (res) =>
  res.status(404).render('404', { titulo: 'Produto não encontrado' });

export function listar(req, res) {
  const produtos = repo.listar();
  res.render('produtos/index', { titulo: 'Produtos', produtos });
}

export function detalhar(req, res) {
  const produto = repo.buscarPorId(req.params.id);
  if (!produto) return naoEncontrado(res);
  res.render('produtos/detalhe', { titulo: produto.nome, produto });
}
```

- Cada **função exportada** é uma **ação** do Controller.
- `listar` busca todos os produtos e escolhe a View `produtos/index`.
- `detalhar` lê o `id` da URL em `req.params` e trata o caso de produto inexistente (404).
- Não há SQL, cálculo de preço ou HTML aqui.

Controller – Objetos de requisição e resposta

Recurso do Express	Uso no Controller	Exemplo
`req.params`	Parâmetros do caminho da URL	`/produtos/:id` → `req.params.id`
`req.query`	Parâmetros da query string	`/produtos?categoria=Monitores`
`req.body`	Dados do formulário ou JSON	`req.body.nome` (requer middleware de parser)
`res.render(view, dados)`	Renderiza uma View com dados	`res.render('produtos/index', { produtos })`
`res.redirect(url)`	Redireciona o navegador (302)	`res.redirect('/produtos/7')`
`res.status(code)`	Define o código de status	`res.status(404).render('404', …)`
`res.json(obj)`	Responde em JSON (APIs)	`res.json(produto)`

Fonte: Express 5.x API Reference. https://expressjs.com/en/5x/api.html

Resumo: quem faz o quê?

Responsabilidade	Model	View	Controller
Validar se o preço é maior que zero	✓		
Ler o id informado na URL			✓
Formatar o preço como R$ 1.599,00		✓	
Decidir o status HTTP (200, 404, 422)			✓
Salvar o produto no armazenamento	✓		
Gerar o HTML da página		✓	
Escolher qual página exibir			✓

Dica: responda antes de ver o gabarito – é uma boa questão de prova!

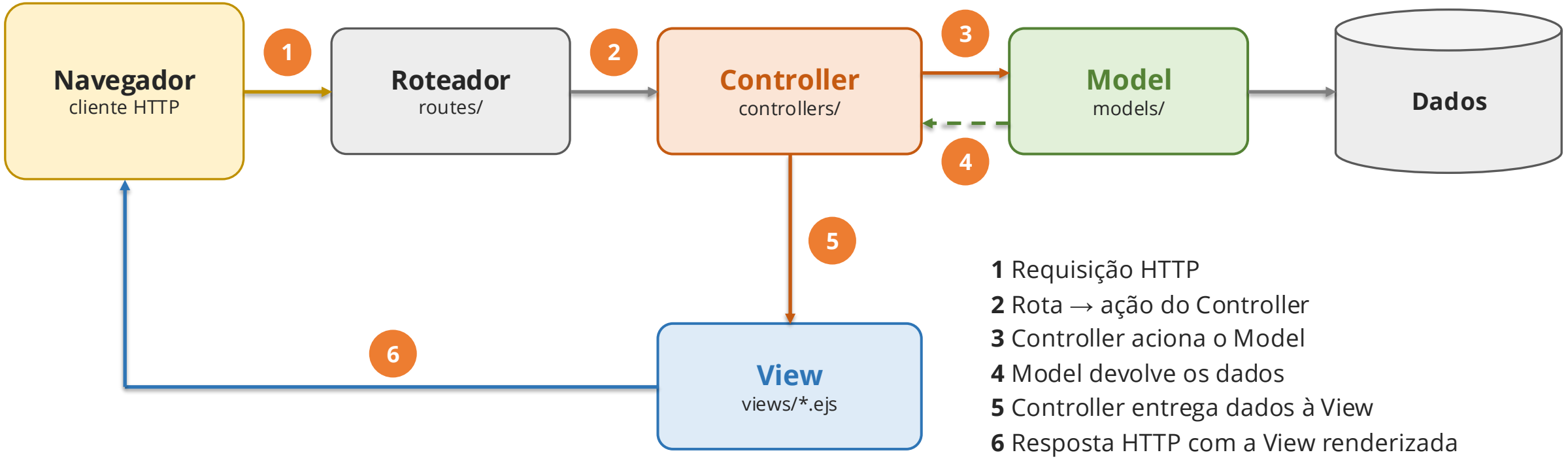
PUC Minas

Padrão MVC

O Ciclo de Vida da Requisição Web no MVC

Prof. Rommel Vieira Carneiro

Ciclo de vida da requisição – visão geral



1 Requisição HTTP
2 Rota → ação do Controller
3 Controller aciona o Model
4 Model devolve os dados
5 Controller entrega dados à View
6 Resposta HTTP com a View renderizada

Passo 1 – O cliente faz a requisição HTTP

O usuário digita uma URL, clica em um link ou envia um formulário. O navegador monta uma **requisição HTTP** com:

- **Método** – GET, POST (formulários HTML aceitam apenas estes dois)
- **Caminho** – o recurso desejado: `/produtos/1`
- **Cabeçalhos** – metadados como `Accept` e `Content-Type`
- **Corpo** – dados do formulário (em POST)

Link: ver detalhes

```
GET /produtos/1 HTTP/1.1
Host: localhost:3000
Accept: text/html
```

Formulário: novo produto

```
POST /produtos HTTP/1.1
Host: localhost:3000
Content-Type: application/x-www-form-urlencoded

nome=Headset+USB&preco=199.90&estoque=3
```

Passo 2 – O roteador (routing)

routes/produtos.routes.js

```
// Roteador: mapeia método HTTP + URL para uma ação do Controller
import { Router } from 'express';
import * as ctrl from '../controllers/produtos.controller.js';

const router = Router();

router.get('/', ctrl.listar);              // GET    /produtos
router.get('/novo', ctrl.novo);            // GET    /produtos/novo
router.post('/', ctrl.criar);              // POST   /produtos
router.get('/:id', ctrl.detalhar);         // GET    /produtos/7
router.get('/:id/editar', ctrl.editar);    // GET    /produtos/7/editar
router.put('/:id', ctrl.atualizar);        // PUT    /produtos/7
router.delete('/:id', ctrl.remover);       // DELETE /produtos/7

export default router;
```

- O **roteador** traduz **método HTTP + caminho** em uma **ação** do Controller.
- `:id` é um **parâmetro de rota**: casa com qualquer valor (`/produtos/7`) e fica disponível em `req.params.id`.
- Em `app.js`, o roteador é montado sob o prefixo `/produtos`.

Desafio → O que acontece se a rota `/:id` for declarada antes de `/novo`?

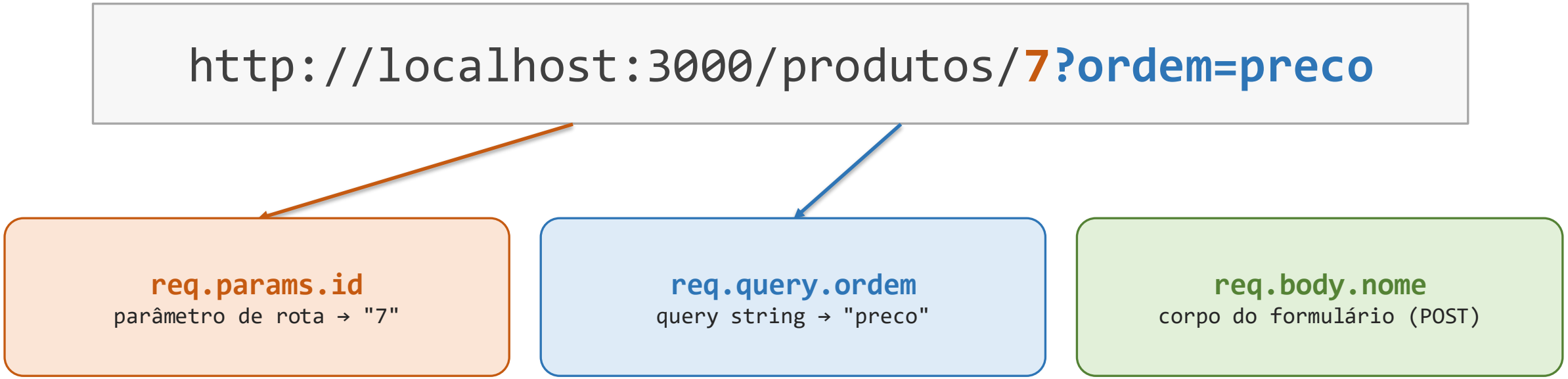
Convenção de rotas para um recurso

Ação	Método	Caminho	Controller	Resposta
Listar	GET	/produtos	listar	View produtos/index
Formulário de criação	GET	/produtos/novo	novo	View produtos/form
Criar	POST	/produtos	criar	Redirect 302 ou form com 422
Detalhar	GET	/produtos/:id	detalhar	View produtos/detalhe ou 404
Formulário de edição	GET	/produtos/:id/editar	editar	View produtos/form
Atualizar	PUT	/produtos/:id	atualizar	Redirect 302 ou form com 422
Excluir	DELETE	/produtos/:id	remover	Redirect 302 para a lista

Convenção inspirada no Ruby on Rails e adotada por Laravel, ASP.NET MVC e outros frameworks: 7 ações cobrem o CRUD completo.

Passo 3 – Processamento do Controller

O Controller captura as informações enviadas pelo cliente em três lugares diferentes:



- Todos os valores chegam como **texto** (`"7"`, `"199.90"`) – a conversão de tipos é responsabilidade do Model (`Number(preco)`).
- `req.body` só é preenchido se houver um middleware de parser: `express.urlencoded()` para formulários e `express.json()` para JSON.

Passos 4 e 5 – Model e dados para a View

controllers/produtos.controller.js

```
export function detalhar(req, res) {
  const produto = repo.buscarPorId(req.params.id);
  if (!produto) return naoEncontrado(res);
  res.render('produtos/detalhe', { titulo: produto.nome, produto });
}
```

views/produtos/detalhe.ejs

```
<h1><%= produto.nome %></h1>
<p>Categoria: <%= produto.categoria %></p>
<p>Estoque: <%= produto.estoque %></p>
```



- **Passo 4** – o Controller chama o Model (`repo.buscarPorId`) e recebe um objeto `Produto` ou `null`.
- **Passo 5** – `res.render('produtos/detalhe', { titulo, produto })` empacota os dados: cada chave do objeto vira uma **variável** disponível no template.
- O nome da View é um caminho relativo à pasta configurada em `app.set('views', …)`: `produtos/detalhe` → `views/produtos/detalhe.ejs`.

Passo 6 – A resposta HTTP

O EJS processa o template, gera o HTML final e o Express envia a **resposta HTTP** com status, cabeçalhos e corpo.

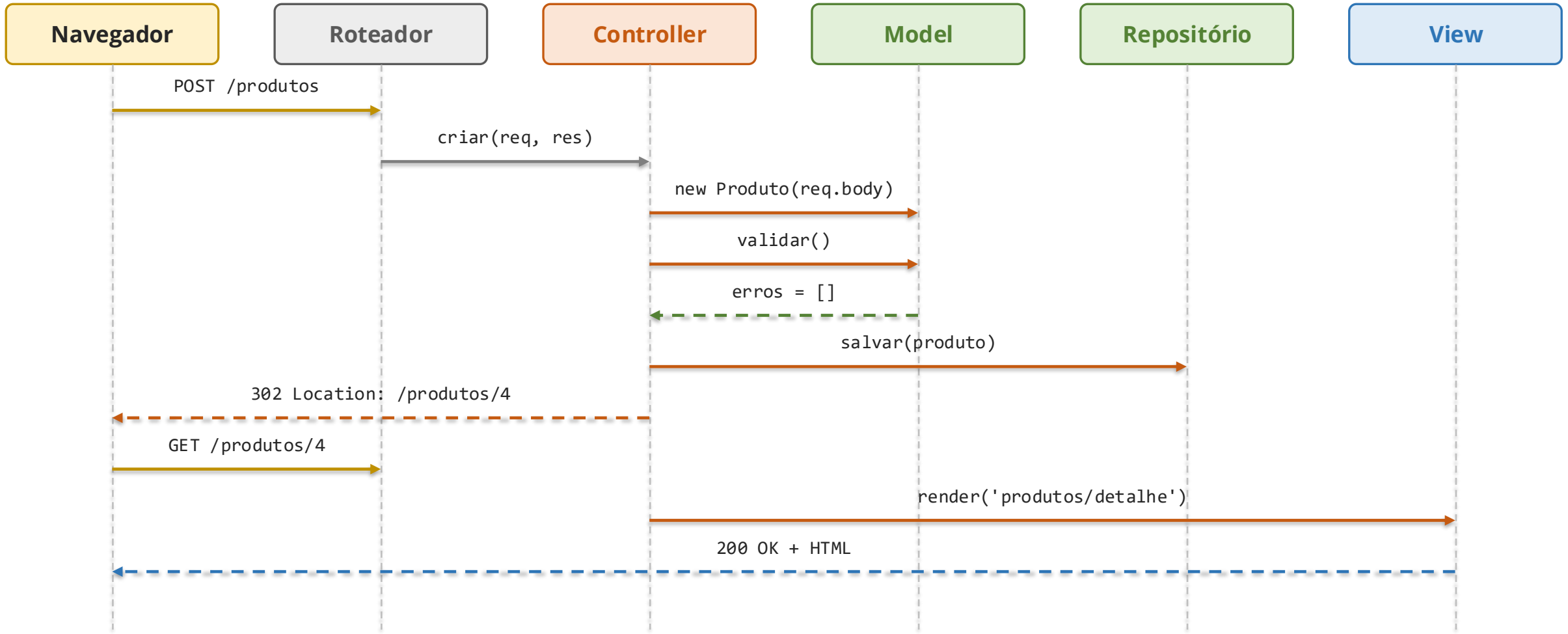
```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 612

<!DOCTYPE html>
<html lang="pt-BR">
<head> ... </head>
<body>
  <h1>Teclado mecânico</h1>
  <p>Categoria: Periféricos</p>
  ...
```



- O navegador interpreta o HTML e dispara novas requisições para CSS, imagens e scripts (arquivos estáticos em `public/`).
- Nenhuma dessas requisições passa pelo Controller: são atendidas por `express.static`.

Fluxo completo: criação de um produto

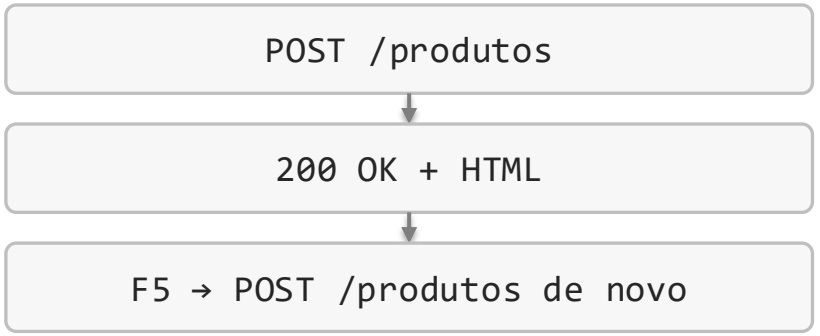


Se `validar()` retornar erros, o Controller responde `422` renderizando o formulário com as mensagens – sem salvar e sem redirecionar.

Padrão Post/Redirect/Get (PRG)

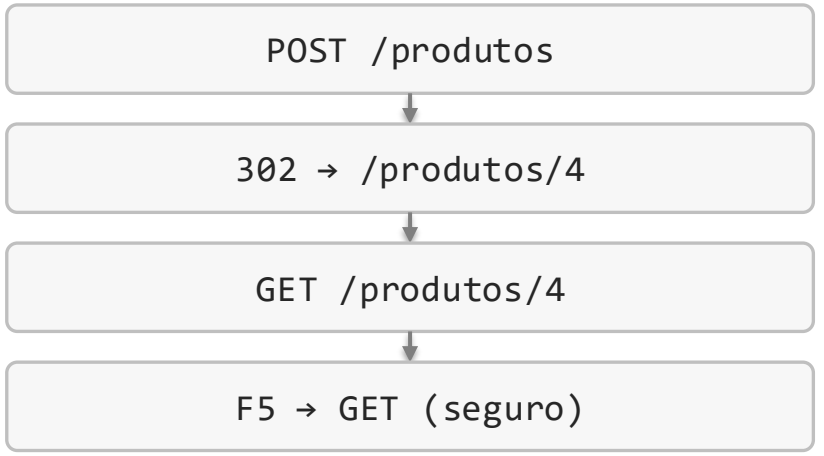
Após um POST bem-sucedido, o Controller **não renderiza** uma página: ele **redireciona** (302) para uma URL de leitura. Assim, recarregar a página (F5) não reenvia o formulário.

Sem PRG



Produto duplicado a cada F5!

Com PRG



Recarregar apenas relê o produto.

HANDS ON – Rastreando o ciclo de vida

app.js (antes das rotas)

```
app.use((req, res, next) => {
  const inicio = Date.now();
  console.log(`[1] ${req.method} ${req.originalUrl}`);
  res.on('finish', () =>
    console.log(`[6] ${res.statusCode} em ${Date.now() - inicio}ms`));
  next();
});
```

```
$ npm run dev
Loja MVC em http://localhost:3000
[1] GET /produtos
[6] 200 em 6ms
```

1. Adicione o middleware de log em `app.js`

2. Inclua `console.log` com os passos **[2]** a **[5]** no roteador, no Controller, no Model e antes do `res.render`

3. Abra as Ferramentas do Desenvolvedor (F12) → aba **Rede**

4. Crie um produto e compare a sequência no terminal com as requisições exibidas no navegador (POST → 302 → GET)

PUC Minas

Padrão MVC

Implementação Prática (Mão na Massa)

Prof. Rommel Vieira Carneiro

Stacks de mercado para MVC

Framework	Linguagem	Template (View)	Convenção MVC
ASP.NET Core MVC	C#	Razor	Imposta pelo framework
Spring MVC / Spring Boot	Java	Thymeleaf	Imposta por anotações
Laravel	PHP	Blade	Imposta pelo framework
Ruby on Rails	Ruby	ERB	Imposta (convention over configuration)
Express	JavaScript	EJS, Pug, Handlebars	Definida pela equipe

Por que Express + EJS nesta unidade?
O Express é minimalista e **não impõe** o MVC: somos nós que criamos as pastas e ligamos as camadas. Isso deixa cada peça do padrão explícita – nos frameworks "opinativos" elas ficam escondidas pela convenção.

Configuração do ambiente

terminal

```
# Pré-requisito: Node.js 24 LTS (ou superior)
$ node --version

# Cria o projeto e ativa ES Modules
$ mkdir loja-mvc && cd loja-mvc
$ npm init -y
$ npm pkg set type=module

# Instala as dependências
$ npm install express ejs method-override

# Scripts de execução
$ npm pkg set scripts.start="node app.js"
$ npm pkg set scripts.dev="node --watch app.js"
$ npm run dev
```

Pacote	Papel
express 5	Servidor HTTP, rotas e middlewares
ejs	Template engine (View)
method-override	PUT e DELETE em formulários HTML

- `node --watch` reinicia o servidor a cada alteração – dispensa o nodemon.
- `type: module` permite usar `import`/`export`.

Estrutura de pastas padrão

```
loja-mvc/
├── app.js
├── package.json
├── models/
│   ├── produto.js
│   ├── produto.repository.js
│   └── categoria.repository.js
├── controllers/
│   ├── produtos.controller.js
│   ├── api.produtos.controller.js
│   ├── api.categorias.controller.js
│   └── problema.js
├── routes/
│   ├── produtos.routes.js
│   └── api.routes.js
├── views/
│   ├── partials/  (cabecalho, rodape)
│   ├── produtos/  (index, detalhe, form)
│   └── 404.ejs
└── public/
    └── css/estilo.css
```

Configuração e inicialização

Model – domínio e persistência

Controller – ações (HTML e API)

Roteamento – URL → ação

View – templates EJS

Arquivos estáticos

A mesma estrutura em outros frameworks

Camada	Express (nosso projeto)	Laravel	ASP.NET Core MVC	Spring Boot
Model	models/	app/Models/	Models/	.../model/, .../repository/
View	views/*.ejs	resources/views/*.blade.php	Views/*.cshtml	resources/templates/*.html
Controller	controllers/	app/Http/Controllers/	Controllers/	.../controller/
Rotas	routes/	routes/web.php	Program.cs + atributos	@GetMapping, @PostMapping

- Os nomes mudam, mas os **papéis são os mesmos** – quem entende o padrão navega com facilidade por qualquer um desses frameworks.
- No **Spring Boot**, que estudaremos a seguir, as camadas são marcadas por anotações: `@Controller`, `@Service` e `@Repository`.

Ponto de entrada: app.js

app.js (1/2)

```
import express from 'express';
import methodOverride from 'method-override';
import path from 'node:path';
import produtosRouter from './routes/produtos.routes.js';
import apiRouter from './routes/api.routes.js';

const app = express();
const raiz = import.meta.dirname;

// Camada View: template engine EJS
app.set('view engine', 'ejs');
app.set('views', path.join(raiz, 'views'));

// Middlewares
app.use(express.static(path.join(raiz, 'public')));
app.use(express.urlencoded({ extended: false })); // formulários
app.use(methodOverride('_method'));               // PUT e DELETE
```

app.js (2/2)

```
// Rotas
app.get('/', (req, res) => res.redirect('/produtos'));
app.use('/produtos', produtosRouter);
app.use('/api', apiRouter);                       // API REST (JSON)

// Rota inexistente (404)
app.use((req, res) => {
  res.status(404).render('404', { titulo: 'Não encontrado' });
});

const PORTA = process.env.PORT ?? 3000;
app.listen(PORTA, () =>
  console.log(`Loja MVC em http://localhost:${PORTA}`));
```

- Configura a **View** (engine EJS), registra os **middlewares** (estáticos, parsers, `method-override`) e monta os **roteadores** sob prefixos de URL.
- `import.meta.dirname` (Node 20.11+) evita erros de caminho relativo; o último middleware trata rotas inexistentes (404).

CRUD – Model: validação

models/produto.js

```
export class Produto {
  // ... construtor

  errosPorCampo() {
    const erros = {};
    if (!this.nome) erros.nome = 'Nome é obrigatório';
    else if (this.nome.length < 3 || this.nome.length > 100)
      erros.nome = 'Nome: de 3 a 100 caracteres';
    if (this.descricao && this.descricao.length > 500)
      erros.descricao = 'Descrição: até 500 caracteres';
    if (this.preco === null) erros.preco = 'Preço é obrigatório';
    else if (!(this.preco > 0)) erros.preco = 'Preço deve ser maior que zero';
    if (this.estoque === null) erros.estoque = 'Estoque é obrigatório';
    else if (!Number.isInteger(this.estoque) || this.estoque < 0)
      erros.estoque = 'Estoque não pode ser negativo';
    return erros;
  }

  validar() {
    return Object.values(this.errosPorCampo());
  }
  // ...
}
```

- `errosPorCampo()` associa cada erro ao campo inválido – formato usado pela API JSON.
- `validar()` devolve só as mensagens – formato usado pelo formulário HTML.
- Uma única fonte de regras para as duas Views.

CRUD – Controller: listar, detalhar e novo

controllers/produtos.controller.js (1/3)

```
// Controller: recebe a requisição, aciona o Model e escolhe a View
import { Produto } from '../models/produto.js';
import { ProdutoRepository as repo } from '../models/produto.repository.js';

const naoEncontrado = (res) =>
  res.status(404).render('404', { titulo: 'Produto não encontrado' });

export function listar(req, res) {
  const produtos = repo.listar();
  res.render('produtos/index', { titulo: 'Produtos', produtos });
}

export function detalhar(req, res) {
  const produto = repo.buscarPorId(req.params.id);
  if (!produto) return naoEncontrado(res);
  res.render('produtos/detalhe', { titulo: produto.nome, produto });
}

export function novo(req, res) {
  const dados = { titulo: 'Novo produto', produto: {}, erros: [] };
  res.render('produtos/form', dados);
}
```

- `naoEncontrado` centraliza a resposta 404, evitando repetição.
- `novo` envia um produto vazio e uma lista de erros vazia – o mesmo formulário servirá para criar e editar.

CRUD – Controller: criar e editar

controllers/produtos.controller.js (2/3)

```
export function criar(req, res) {
  const produto = new Produto(req.body);
  const erros = produto.validar();
  if (erros.length > 0) {
    const dados = { titulo: 'Novo produto', produto, erros };
    return res.status(422).render('produtos/form', dados);
  }
  repo.salvar(produto);
  res.redirect(`/produtos/${produto.id}`);
}

export function editar(req, res) {
  const produto = repo.buscarPorId(req.params.id);
  if (!produto) return naoEncontrado(res);
  const dados = { titulo: 'Editar produto', produto, erros: [] };
  res.render('produtos/form', dados);
}
```

- Com erros → status **422** e o formulário volta com os dados digitados e as mensagens.
- Sem erros → salva e **redireciona** (PRG).
- `editar` reaproveita o mesmo formulário de `novo`.

CRUD – Controller: atualizar e remover

controllers/produtos.controller.js (3/3)

```
export function atualizar(req, res) {
  const existente = repo.buscarPorId(req.params.id);
  if (!existente) return naoEncontrado(res);
  const produto = new Produto({ ...req.body, id: existente.id });
  const erros = produto.validar();
  if (erros.length > 0) {
    const dados = { titulo: 'Editar produto', produto, erros };
    return res.status(422).render('produtos/form', dados);
  }
  repo.salvar(produto);
  res.redirect(`/produtos/${produto.id}`);
}

export function remover(req, res) {
  repo.remover(req.params.id);
  res.redirect('/produtos');
}
```

- Em `atualizar`, o `id` vem da URL – nunca do formulário, que pode ser adulterado.
- Antes de atualizar, verifica se o produto existe (404).
- `remover` redireciona para a lista após excluir.

CRUD – View: formulário de criação e edição

views/produtos/form.ejs

```
<%- include('../partials/cabecalho') %>
<h1><%= titulo %></h1>

<% if (erros.length > 0) { %>
  <ul class="erros">
    <% erros.forEach(erro => { %><li><%= erro %></li><% }) %>
  </ul>
<% } %>

<% const acao = produto.id ? `/produtos/${produto.id}?_method=PUT`
                           : '/produtos' %>
<form action="<%= acao %>" method="POST">
  <label>Nome <input name="nome" value="<%= produto.nome ?? '' %>"></label>
  <label>Descrição
    <textarea name="descricao"><%= produto.descricao ?? '' %></textarea></label>
  <label>Preço <input name="preco" type="number" step="0.01"
                      value="<%= produto.preco ?? '' %>"></label>
  <label>Estoque <input name="estoque" type="number"
                        value="<%= produto.estoque ?? 0 %>"></label>
  <label>Categoria <input name="categoria"
                          value="<%= produto.categoria ?? '' %>"></label>
  <button type="submit">Salvar</button>
</form>
<%- include('../partials/rodape') %>
```

- Exibe os **erros de validação** enviados pelo Controller.
- Um único template para criar e editar: a presença de `produto.id` define a ação.
- `?_method=PUT` indica ao `method-override` o método real da requisição.
- `?? ''` evita exibir `undefined` quando o campo está vazio.

CRUD – View: detalhe e exclusão

views/produtos/detalhe.ejs

```
<%- include('../partials/cabecalho') %>
<h1><%= produto.nome %></h1>
<% if (produto.descricao) { %><p><%= produto.descricao %></p><% } %>
<p>Categoria: <%= produto.categoria %></p>
<p>Preço: <%= produto.preco.toLocaleString('pt-BR',
                { style: 'currency', currency: 'BRL' }) %></p>
<p>Estoque: <%= produto.estoque %> unidade(s)</p>

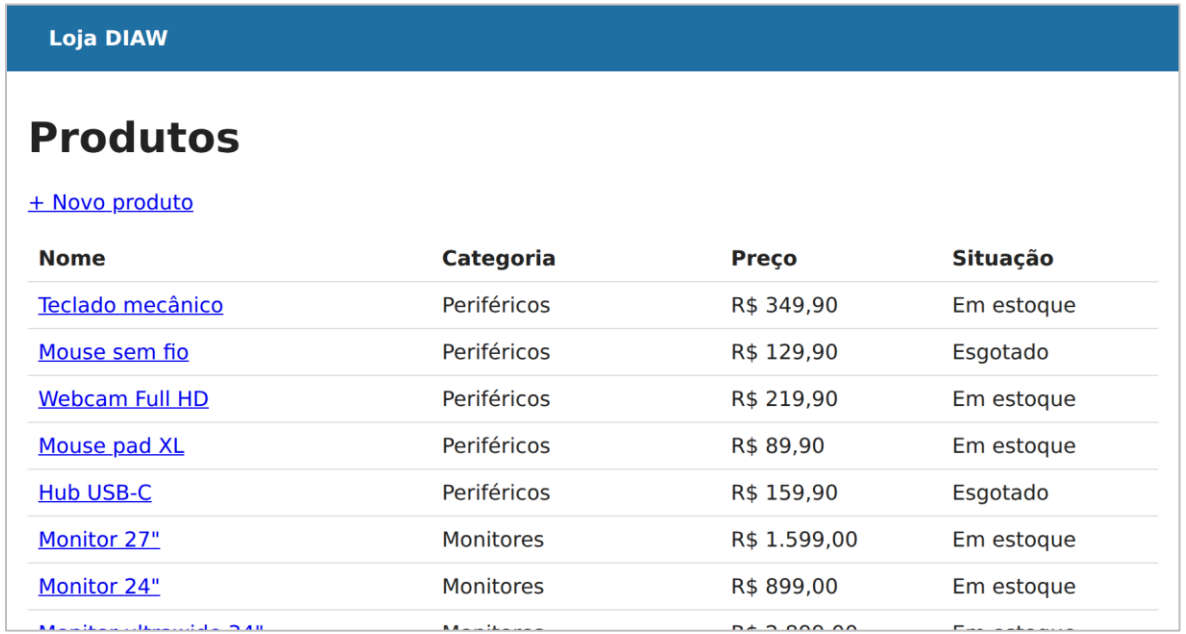
<a href="/produtos/<%= produto.id %>/editar">Editar</a>
<form action="/produtos/<%= produto.id %>?_method=DELETE" method="POST">
  <button type="submit">Excluir</button>
</form>
<a href="/produtos">Voltar</a>
<%- include('../partials/rodape') %>
```

- A exclusão usa um **formulário** – links (`<a>`) só geram GET.
- GET nunca deve alterar dados: é um método **seguro** (safe) do HTTP.

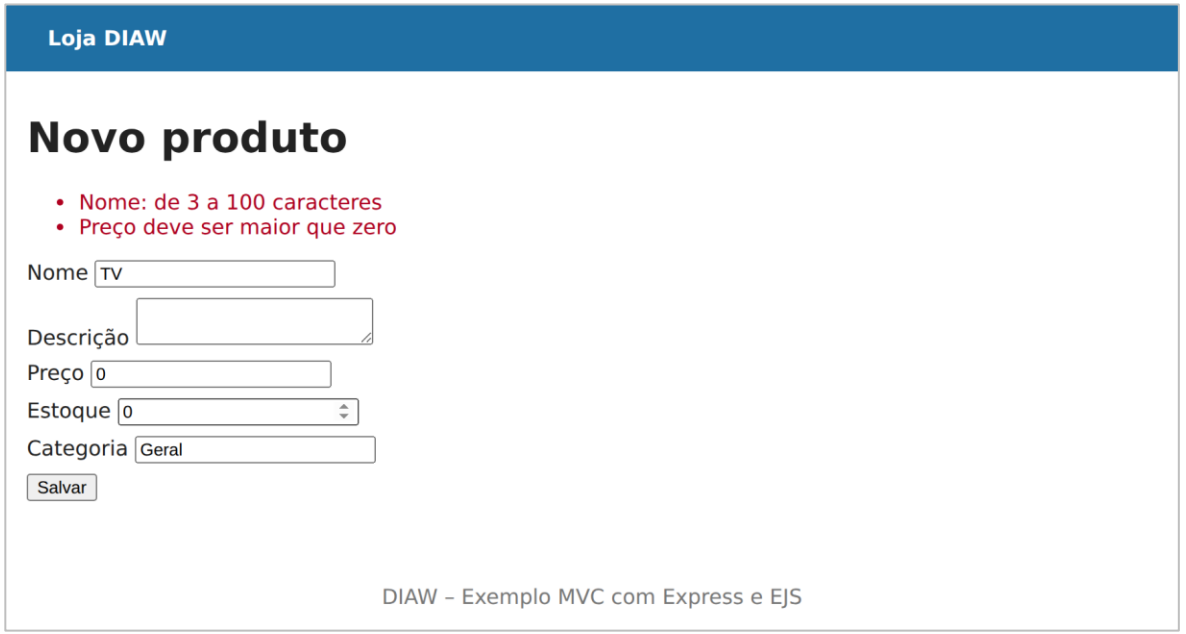
Problema Formulários HTML só enviam **GET** e **POST**. Como enviar PUT e DELETE?

Solução O middleware `method-override` lê `?_method=DELETE` e troca o método da requisição **antes** do roteamento.

Resultado



Listagem – GET /produtos



Validação do Model – POST /produtos → 422

As mensagens de erro são geradas pelo **Model** (`validar()`), decididas pelo **Controller** (status 422) e exibidas pela **View** (`form.ejs`) – cada camada no seu papel.

Desafios

1 Filtro por categoria

Altere `listar` para aceitar `/produtos?categoria=Monitores`. Onde fica o filtro: Controller ou repositório?

2 Categorias no formulário

Troque o campo de texto da categoria por um `<select>` alimentado pelo `CategoriaRepository`.

3 Promoção

Exiba o preço com 10% de desconto nos produtos com mais de 10 unidades, sem colocar a regra na View.

4 Helper de formatação

Evite repetir `toLocaleString` nas Views usando `app.locals` para registrar uma função `moeda()`.

PUC Minas

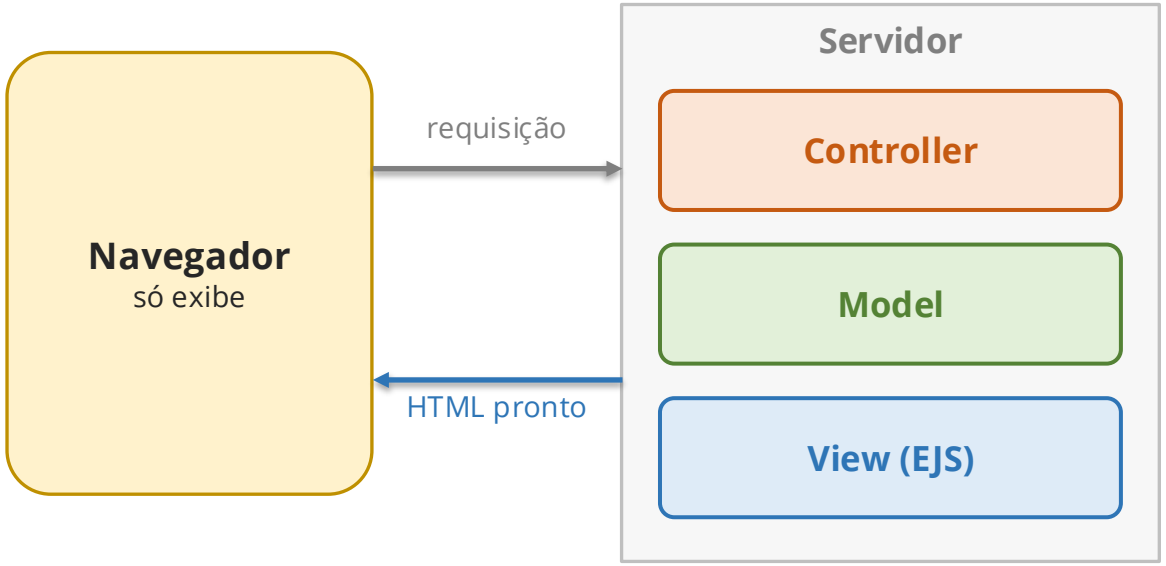
Padrão MVC

MVC no Contexto Moderno

Prof. Rommel Vieira Carneiro

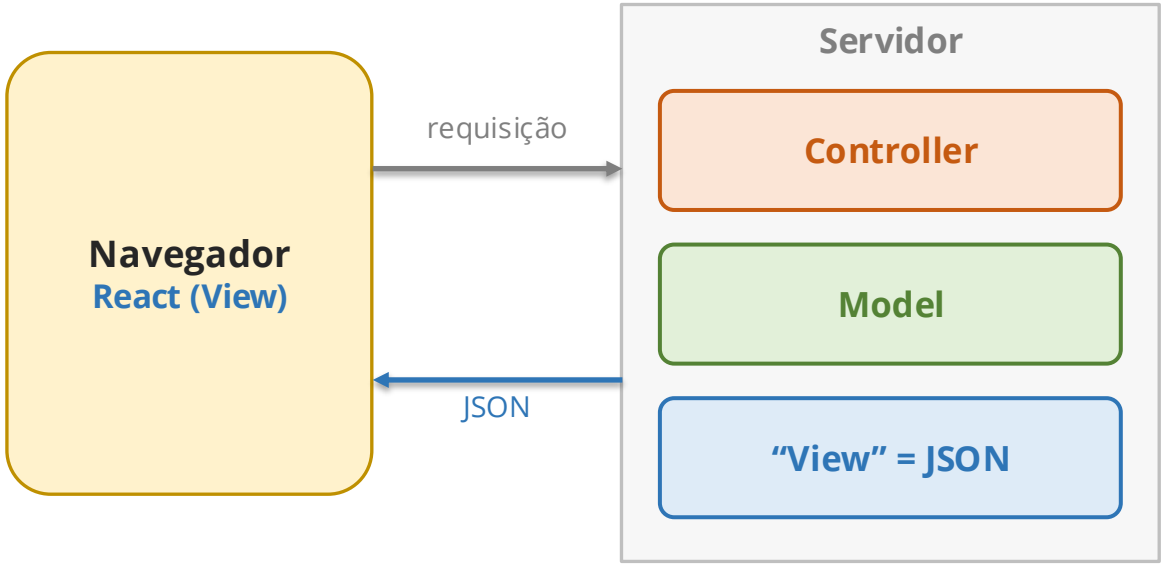
SSR vs. SPA: onde o HTML é gerado?

MVC tradicional – SSR



O servidor gera a página completa a cada requisição.

API REST + SPA



O servidor entrega dados; o navegador monta a interface.

No modelo SPA, o MVC do back-end continua existindo – mas a responsabilidade de apresentação migra para o navegador.

SSR vs. SPA – comparação

Critério	SSR (MVC tradicional)	SPA (API + front-end)
Onde o HTML é gerado	No servidor, a cada requisição	No navegador, com JavaScript
Primeira carga	Rápida – HTML pronto	Mais lenta – baixa o JavaScript antes
Navegação	Recarrega a página inteira	Atualiza só o necessário
SEO e acessibilidade	Favorecidos por padrão	Exigem cuidados adicionais
Clientes atendidos	Principalmente navegadores	Web, mobile e outras aplicações
Exemplos	Express + EJS, Rails, Laravel	React, Vue, Angular + API REST

Tendência atual: abordagens híbridas
Frameworks como o **Next.js** renderizam no servidor e "hidratam" a página no navegador – tema da unidade de React.

MVC em APIs REST: a View vira JSON

controllers/api.produtos.controller.js (trecho)

```
// DTO de resposta: o que a API expõe de um produto
const paraResposta = (p) => ({
  id: String(p.id),
  nome: p.nome,
  descricao: p.descricao,
  preco: p.preco,
  estoque: p.estoque,
  disponivel: p.disponivel,
  categoriaId: categorias.buscarPorNome(p.categoria)?.id ?? null,
  categoria: p.categoria,
});

const naoEncontrado = (res, id) =>
  problema(res, 404, `Produto ${id} não encontrado`);

export function detalhar(req, res) {
  const produto = produtos.buscarPorId(req.params.id);
  if (!produto) return naoEncontrado(res, req.params.id);
  res.json(paraResposta(produto));
}
```

- Mesmo **Model** e mesmo **repositório** – nenhuma regra foi duplicada.
- A "View" passa a ser a **representação JSON**: `paraResposta` define o que a API expõe (um DTO).
- Resultados comunicados pelo **status HTTP**: 200, 201, 204, 400, 404.
- A API completa segue o **contrato** compartilhado com a versão em Spring Boot (unidade de React/Next.js).

Uma ação, várias representações

Com `res.format()`, o Controller consulta o cabeçalho **Accept** da requisição e escolhe a View adequada – **negociação de conteúdo** do HTTP.

controllers/produtos.controller.js

```
export function listar(req, res) {
  const produtos = repo.listar();
  res.format({
    html: () =>
      res.render('produtos/index', { titulo: 'Produtos', produtos }),
    json: () => res.json(produtos),
  });
}
```

```
$ curl localhost:3000/produtos \
    -H "Accept: application/json"
[{"id":1,"nome":"Teclado mecânico",
  "preco":349.9, ...}]
```

- Navegadores enviam `Accept: text/html` → recebem a página EJS.
- Clientes de API enviam `Accept: application/json` → recebem JSON.
- Demonstra a ideia central do MVC: **trocar a View sem tocar no Model**.

MVC no front-end: componentes React

BotaoComprar.jsx

```
function BotaoComprar({ produto }) {
  const [qtd, setQtd] = useState(0);

  function adicionar() {
    if (qtd < produto.estoque) setQtd(qtd + 1);
  }

  return (
    <div>
      <h3>{produto.nome}</h3>
      <button onClick={adicionar}>
        Adicionar ({qtd})
      </button>
    </div>
  );
}
```

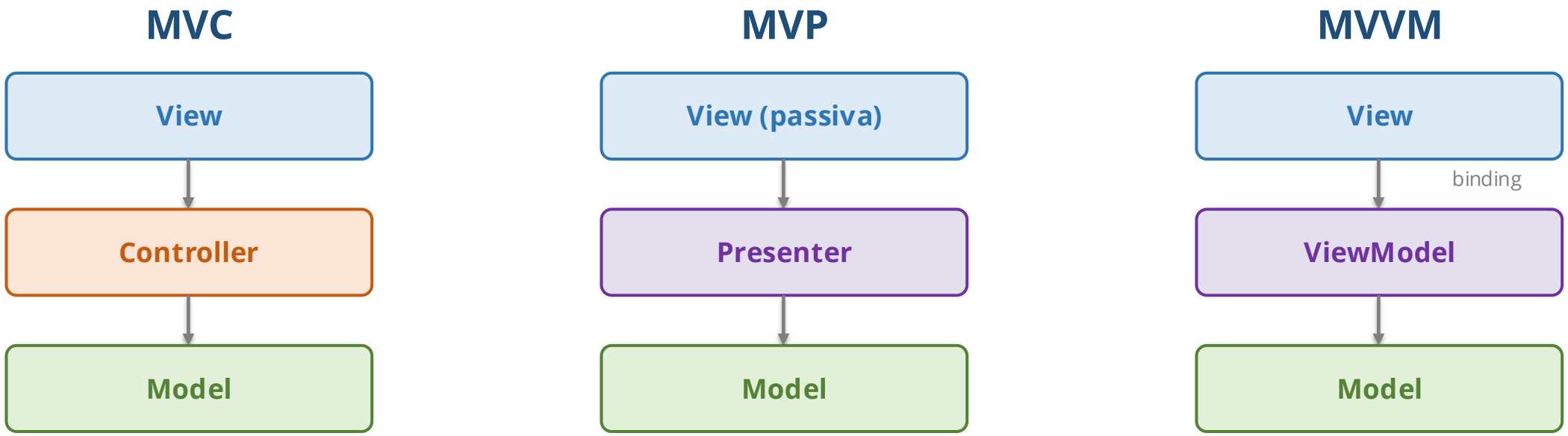
Estado → papel de Model

Evento → papel de Controller

JSX → papel de View

Atenção: o React se define como biblioteca de interface, não como framework MVC – a analogia ajuda a entender, mas o componente reúne os três papéis.

Variações do MVC: MVP e MVVM



Controller recebe a entrada e escolhe a View

Presenter atualiza a View via interface

Data binding sincroniza View e ViewModel

	MVC	MVP	MVVM
Onde é comum	Web no servidor: Express, Spring, Rails	Windows Forms, Android (clássico)	WPF, .NET MAUI, Vue, Angular

Fat Model, Skinny Controller

✗ Controller gordo

```
export function criar(req, res) {
  const { nome, preco, estoque } = req.body;
  const erros = [];
  if (!nome || nome.trim().length < 3)
    erros.push('Nome deve ter ao menos 3 caracteres');
  if (!(Number(preco) > 0))
    erros.push('Preço deve ser maior que zero');
  if (!Number.isInteger(Number(estoque)))
    erros.push('Estoque deve ser inteiro');
  if (erros.length > 0) return res.status(422)
    .render('produtos/form', { ... });
  repo.salvar({ nome: nome.trim(), ... });
  res.redirect('/produtos');
}
```

✓ Controller magro + Model rico

```
export function criar(req, res) {
  const produto = new Produto(req.body);
  const erros = produto.validar();
  if (erros.length > 0) {
    const dados = { titulo: 'Novo produto', produto, erros };
    return res.status(422).render('produtos/form', dados);
  }
  repo.salvar(produto);
  res.redirect(`/produtos/${produto.id}`);
}
```

Regras de negócio concentradas no **Model**; o Controller fica restrito a HTTP – ler a entrada, chamar o Model e escolher a resposta. A validação passa a valer também para a API.

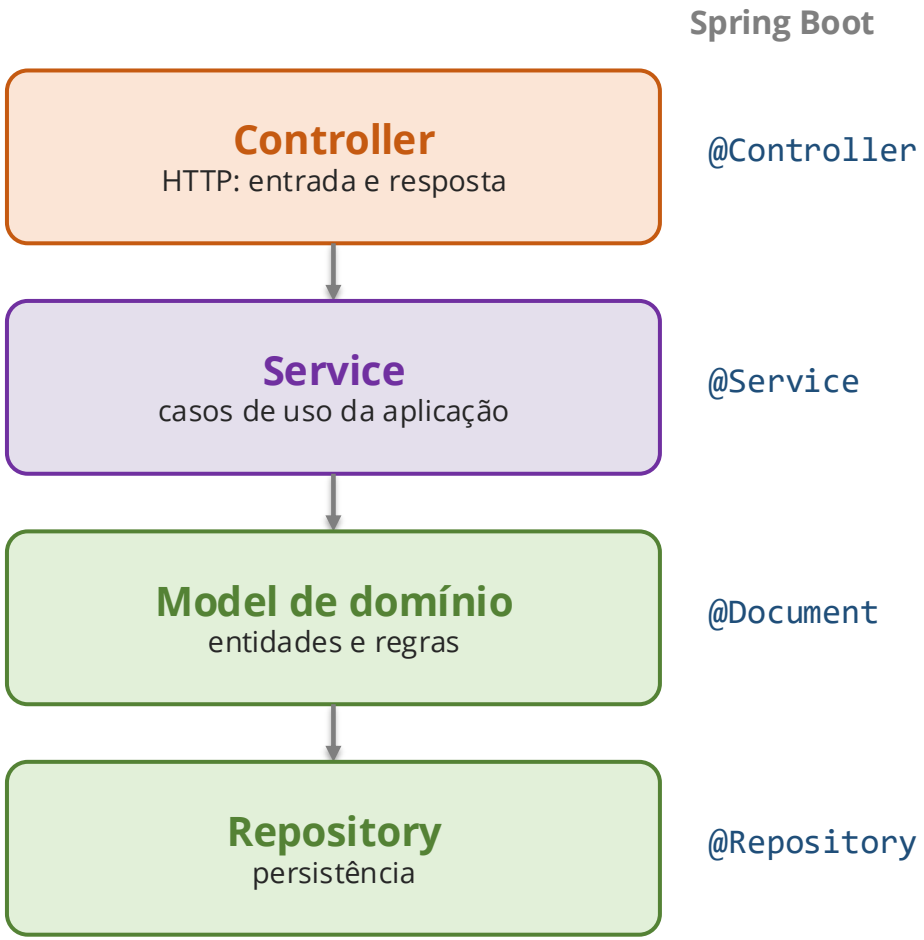
Além do Fat Model: a camada de serviço

Um Model "gordo demais" vira um **God Object**. Quando uma regra envolve **vários objetos** – fechar um pedido, aplicar cupom e baixar estoque – ela não pertence a um único Model.

Solução: uma camada de serviço (Service Layer)

services/pedido.service.js (ilustrativo)

```
export function fecharPedido(clienteId, itens, cupom) {
  const pedido = new Pedido(clienteId, itens);
  pedido.aplicarCupom(cupomRepo.buscar(cupom));
  for (const item of itens) {
    const produto = produtoRepo.buscarPorId(item.produtoId);
    produto.baixarEstoque(item.qtd);
  }
  return pedidoRepo.salvar(pedido);
}
```



Fonte: FOWLER, M. Patterns of Enterprise Application Architecture. Addison-Wesley, 2002 – Service Layer e Domain Model.

Boas práticas – síntese

Model

- Concentra regras e validações
- Não conhece HTTP nem HTML
- Persistência isolada em repositórios

View

- Só exibe e formata
- Escapa dados com `<%= %>`
- Reutiliza trechos com partials

Controller

- Enxuto: entrada → Model → resposta
- Usa status HTTP corretos (201, 404, 422)
- Aplica o padrão PRG após POST

E quando o sistema cresce...
Introduza uma camada de serviço para casos de uso que envolvem vários objetos, e mantenha o mesmo Model atendendo páginas HTML e APIs JSON.

Referências

- GAMMA, Erich et al. **Padrões de projeto**: soluções reutilizáveis de software orientado a objetos. Porto Alegre: Bookman, 2000.
- FOWLER, Martin. **Patterns of Enterprise Application Architecture**. Boston: Addison-Wesley, 2002.
- KRASNER, Glenn E.; POPE, Stephen T. A cookbook for using the model-view controller user interface paradigm in Smalltalk-80. **Journal of Object-Oriented Programming**, v. 1, n. 3, p. 26-49, 1988.
- REENSKAUG, Trygve. **The original MVC reports**. Palo Alto: Xerox PARC, 1979. Disponível em: https://folk.universitetetioslo.no/trygver
- POTEL, Mike. **MVP: Model-View-Presenter – the Taligent programming model for C++ and Java**. Taligent Inc., 1996.
- EXPRESS. **Express 5.x – API Reference**. Disponível em: https://expressjs.com/en/5x/api.html
- EJS. **Embedded JavaScript templating**. Disponível em: https://ejs.co
- MDN WEB DOCS. **Express Tutorial Part 4: Routes and controllers**. Disponível em: https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Server-side/Express_Nodejs/routes

PUC Minas

Obrigado!

Prof. Rommel Vieira Carneiro